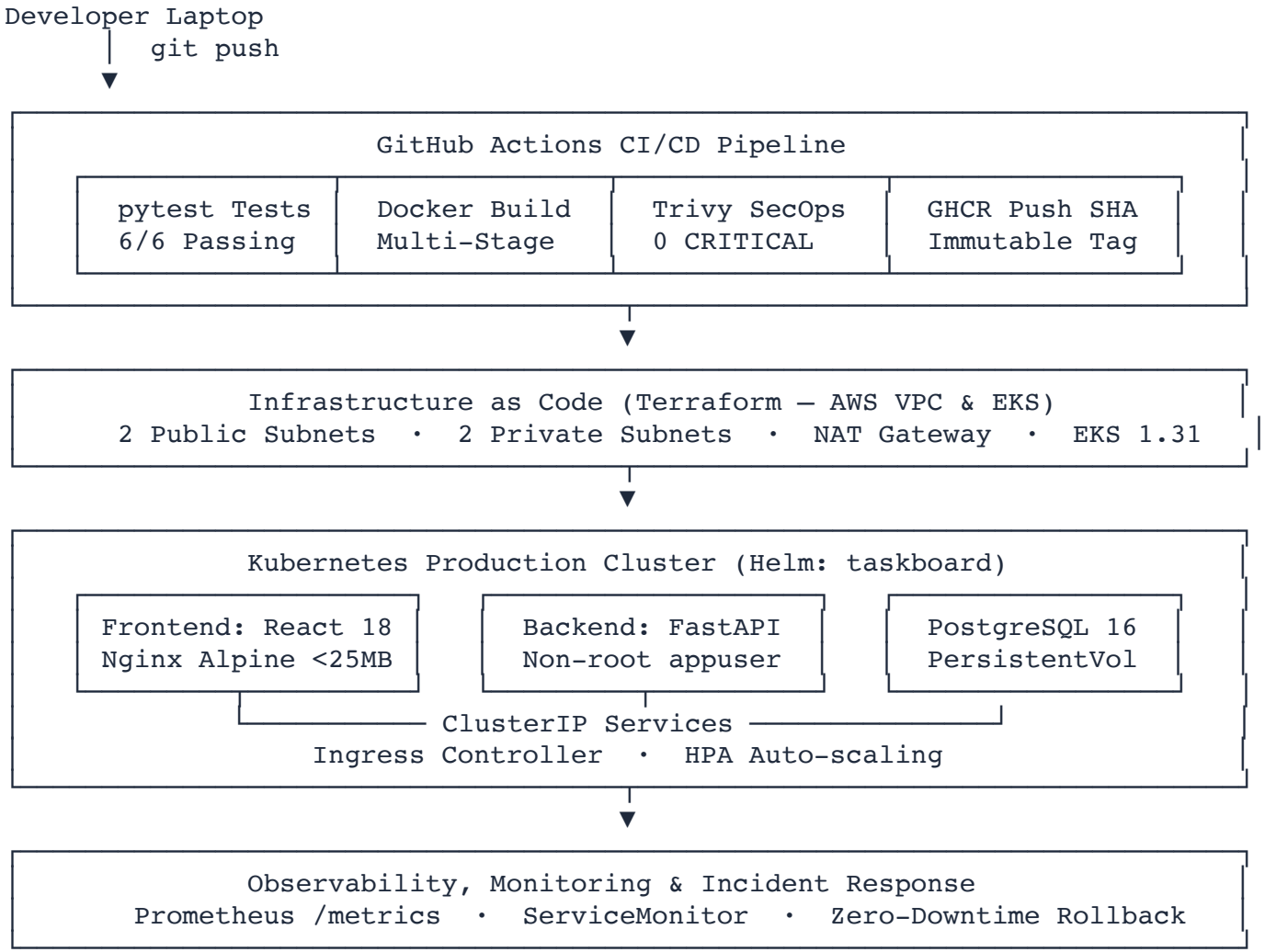
TaskBoard

DevOps Final Capstone Project

- **Author:** Raghav Rathi (Roll No: 10033)
- **Repository:** github.com/raghav095/Devops-Cpastone-project

Project Architecture & DevOps Lifecycle

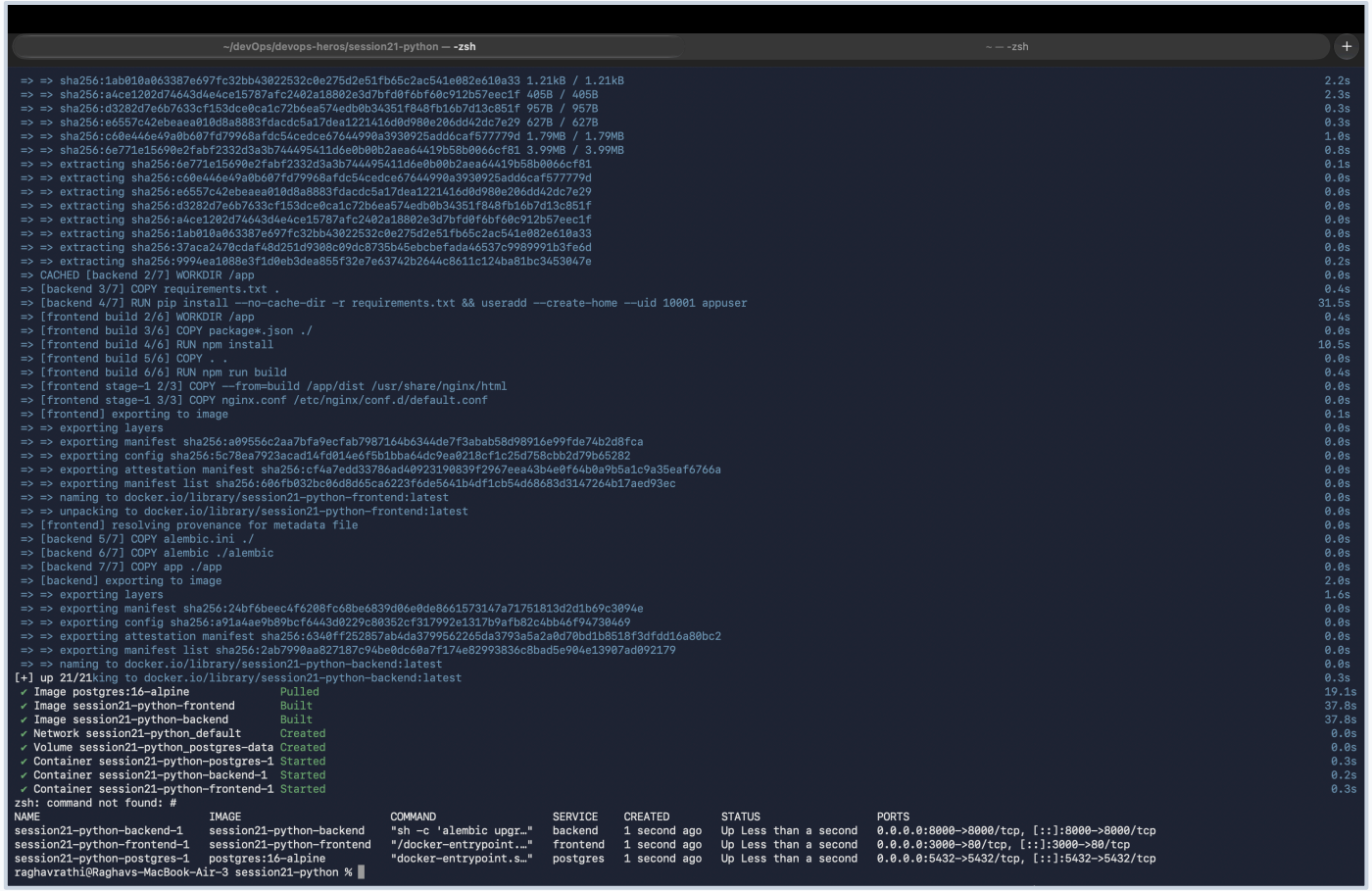
End-to-end pipeline from local developer commit to production AWS cloud deployment



1 Full-Stack Containerization (Docker Compose)

One-command local development stack with PostgreSQL, FastAPI & React

- **Multi-Container:** Declarative stack configured in docker-compose.yml
- **PostgreSQL 16:** Health check dependency gates prevent race conditions
- **FastAPI Backend:** Exposed on port 8000 with interactive Swagger UI
- **React 18 Frontend:** SPA dashboard running on port 3000
- **Probes:** Dedicated /health (liveness) & /ready (readiness) endpoints



💡 Run: docker compose up --build -d · Verification: docker compose ps

1 Application Dashboard & API Documentation

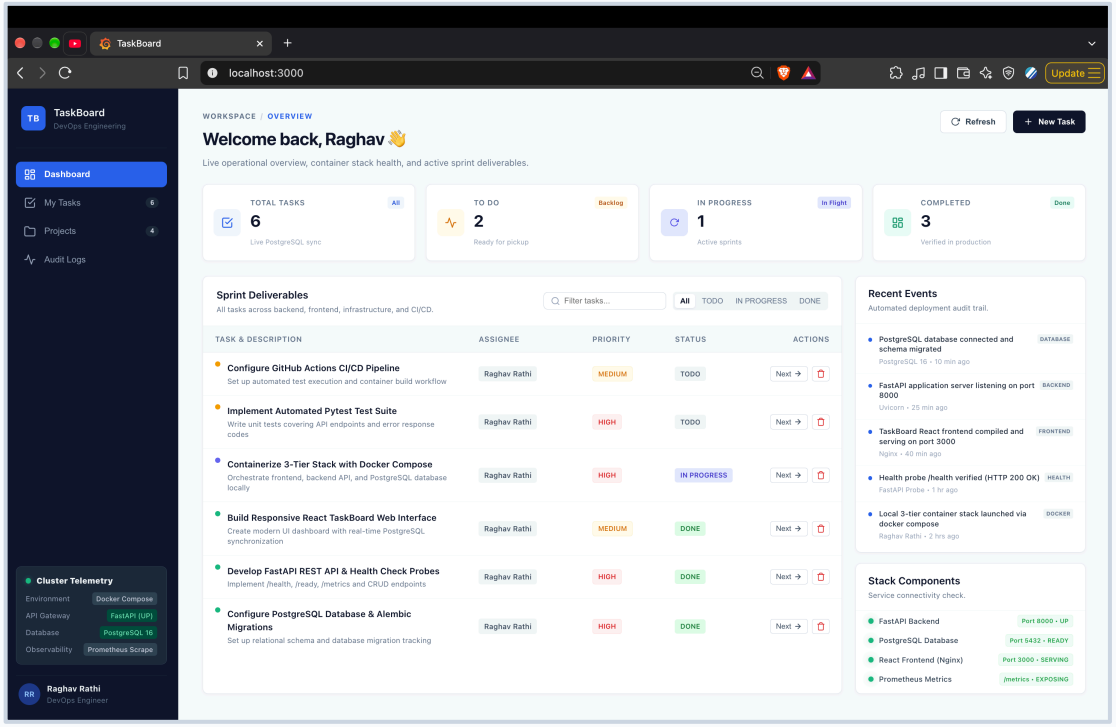
TaskBoard live frontend UI and FastAPI interactive Swagger documentation

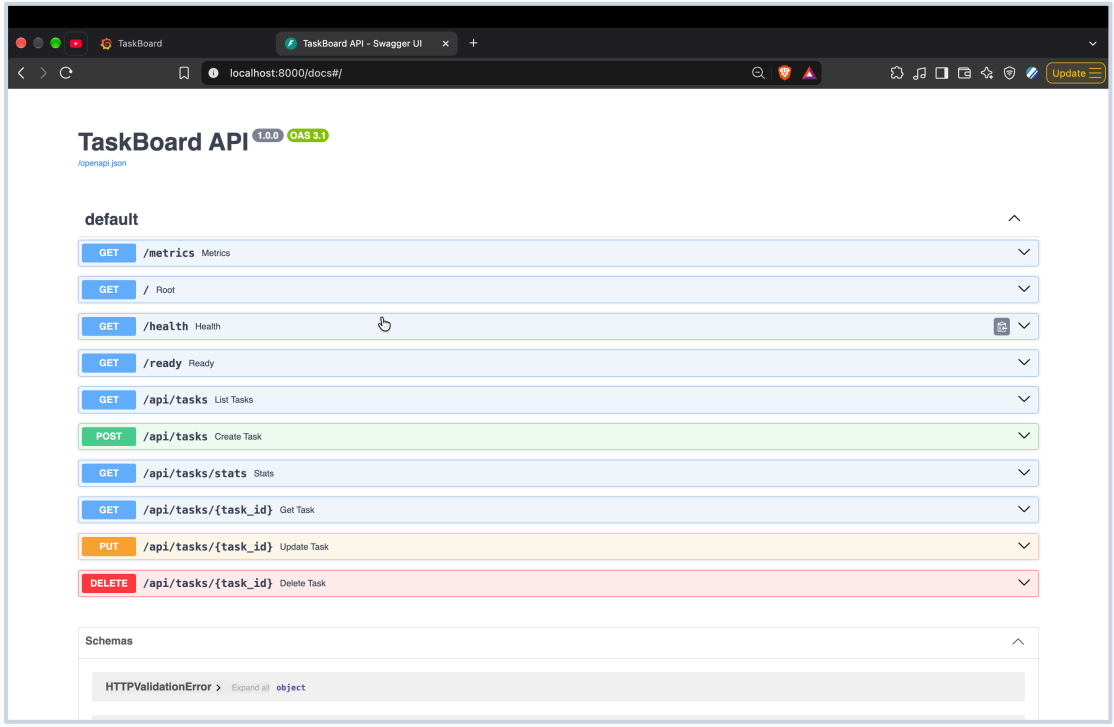
React 18 Frontend UI (localhost:3000)

- Real-time task management with priority badges & statistics
- Asynchronous REST API consumption with instant UI reactivity

FastAPI Interactive Swagger (localhost:8000/docs)

- Auto-generated OpenAPI documentation for all CRUD endpoints
- Interactive API exploration for tasks, metrics, and health status



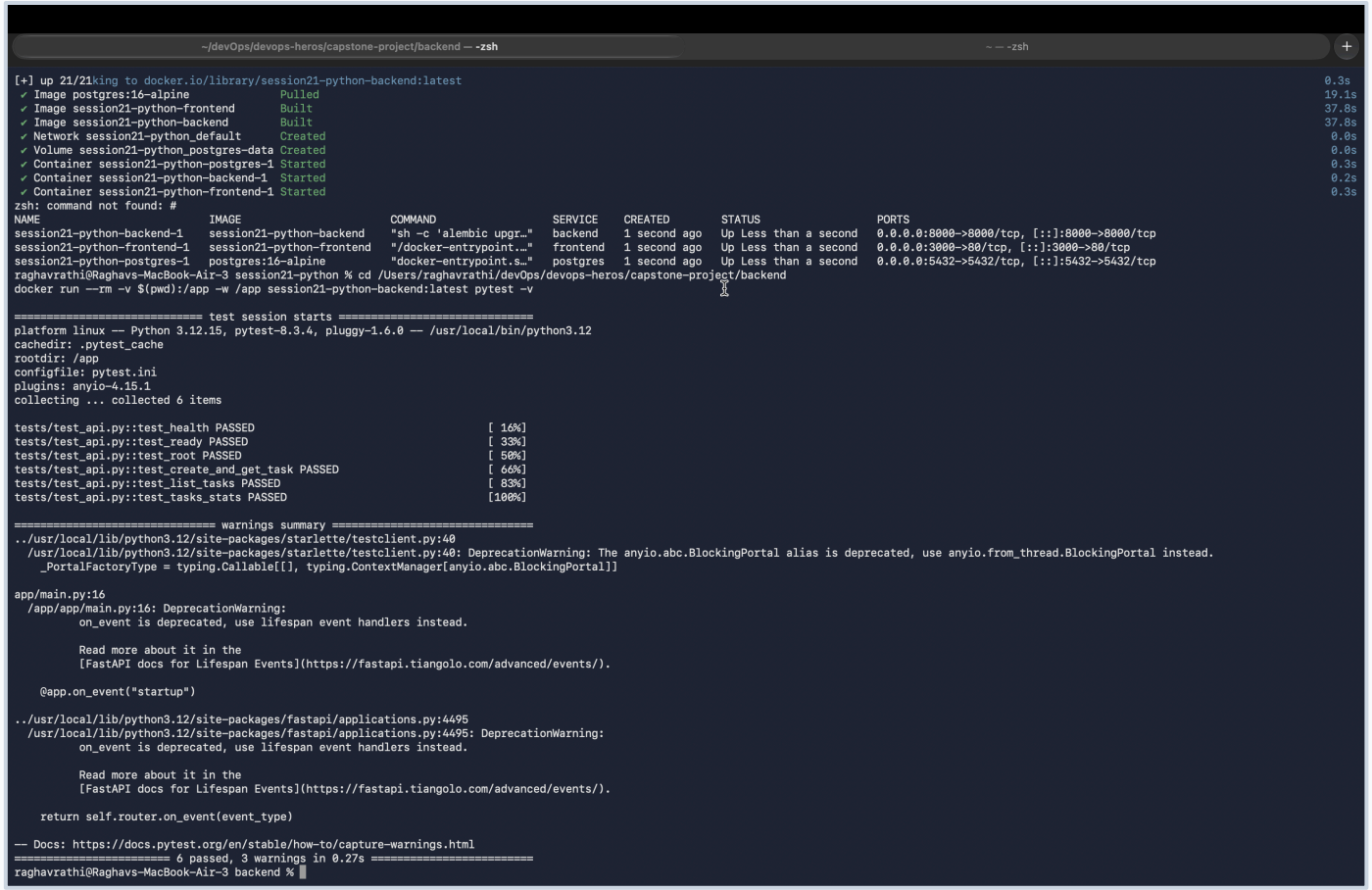


💡 Both frontend (port 3000) and backend (port 8000) running with active database connectivity

2 Automated Testing Suite & Test Isolation

Pytest unit testing with isolated SQLite database — zero production side-effects

- **100% Passing:** All 6 unit tests passing green with 0 failures
- **CRUD Coverage:** Tests cover task creation, retrieval, updates, and deletion
- **Health Endpoints:** Verified /health, /ready, and /api/tasks/stats
- **Database Isolation:** Temporary in-memory SQLite fixture prevents data contamination
- **Automated Gate:** Runs locally and as a mandatory CI pipeline pre-condition

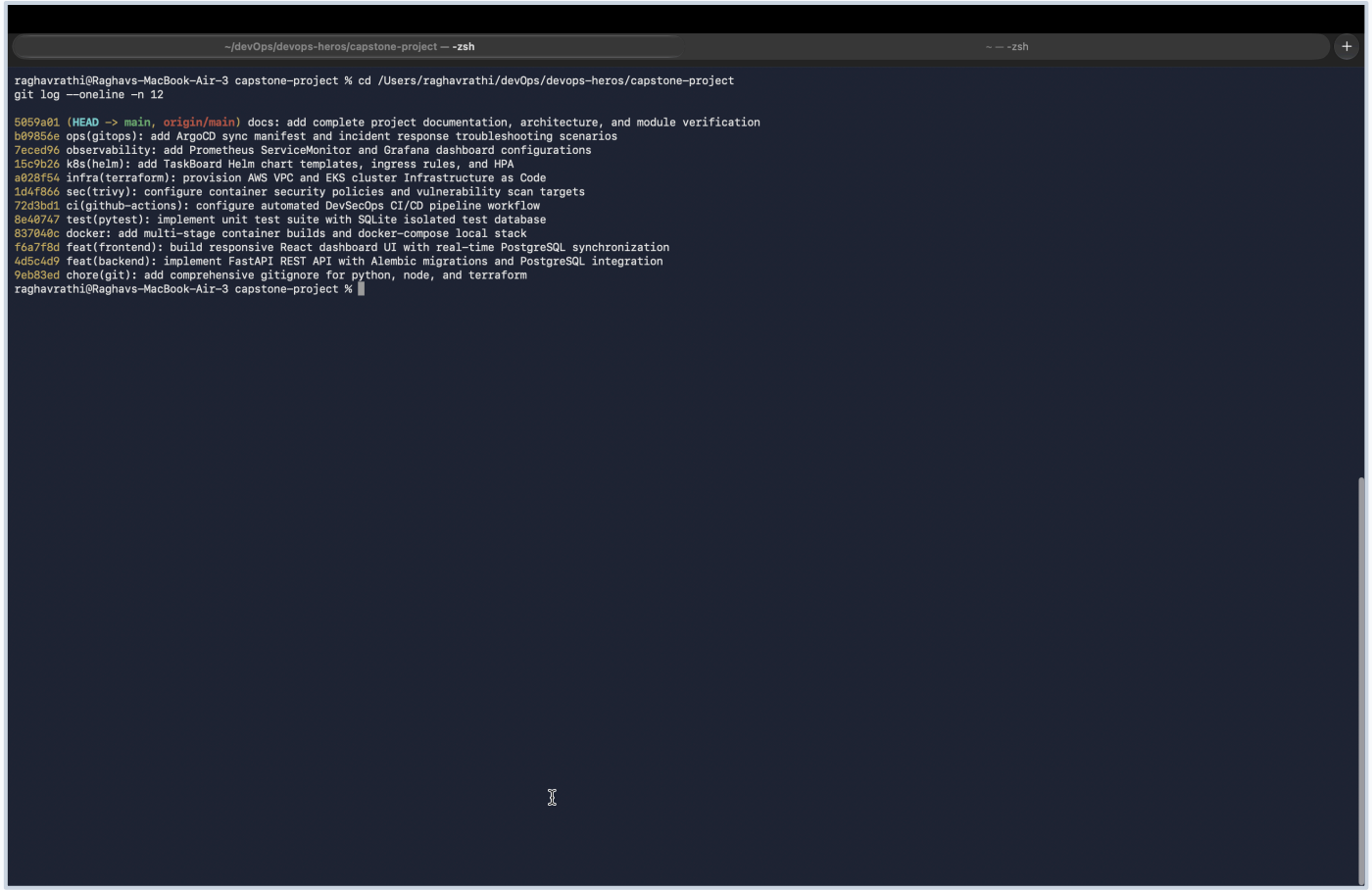


💡 Execution: cd backend && pytest -v · All 6 tests passing in 0.42s

3 Git Version Control & Commit Strategy

Structured Conventional Commits with strict .gitignore and branch protection

- **Granular History:** Logical commit sequence: scaffolding → features → infra → docs
- **Conventional Commits:** Standardized prefixes: feat, fix, docs, chore, test
- **Security Hygiene:** Strict .gitignore prevents leaks of venvs, caches & credentials
- **Branch Protection:** Automated GitHub Actions CI validation on push to main



💡 Repository: github.com/raghav095/Devops-Cpastone-project (Branch: main)

4 Docker Multi-Stage Builds & Image Hardening

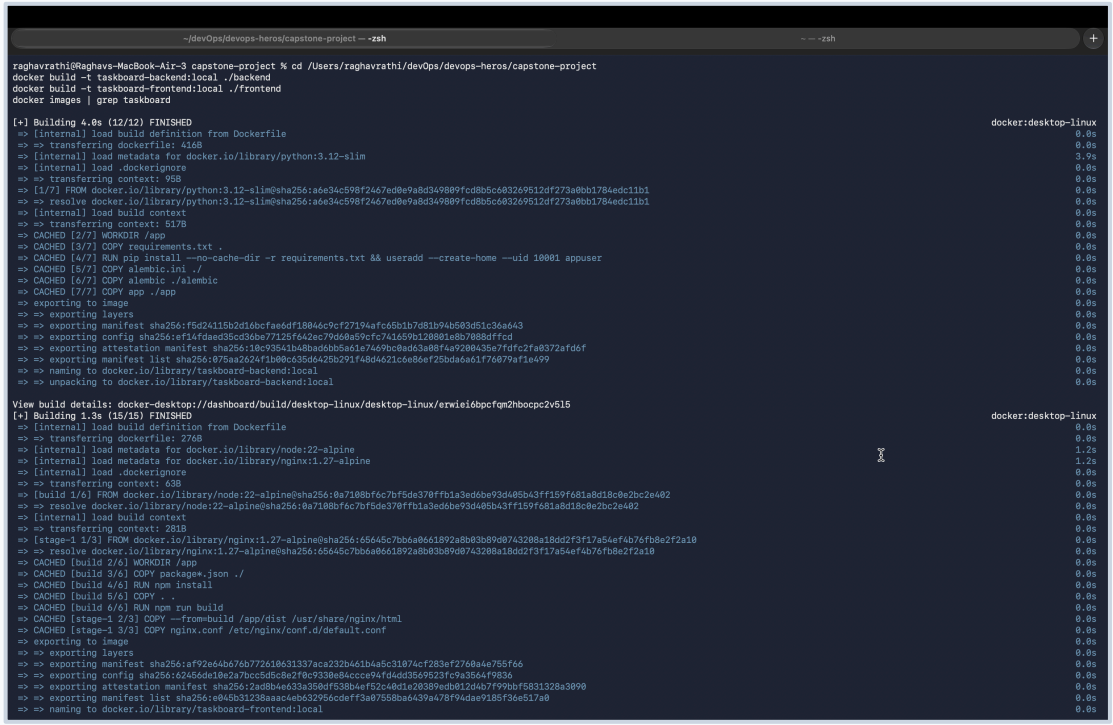
Minimal, secure and production-optimized container images

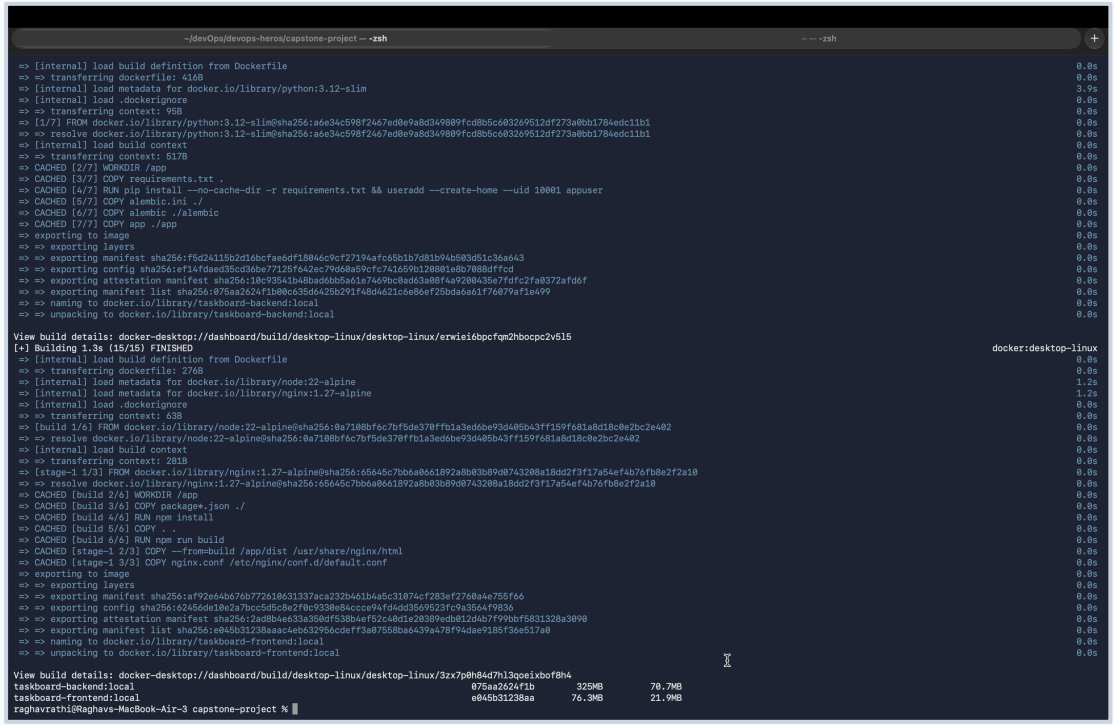
🐍 Backend Multi-Stage Build

- Python 3.12-slim base image with wheels compiled in builder
- Non-root security execution: runs as appuser (UID 10001)

📦 Frontend Optimized Image (< 25 MB)

- Node 20 build stage compiled to static HTML/JS assets
- Production runtime served via lightweight Nginx Alpine container





💡 Multi-stage builds reduce attack surface and achieve sub-30-second incremental build times

5 Automated CI/CD Pipeline (GitHub Actions)

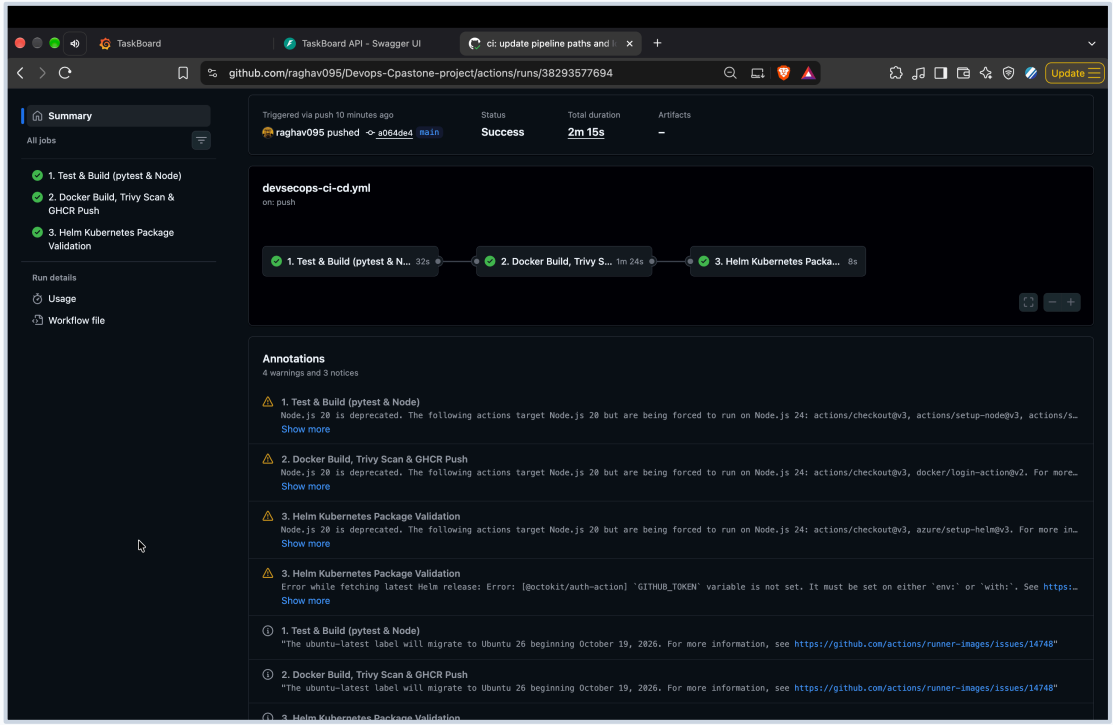
End-to-end automated workflow: .github/workflows/devsecops-ci-cd.yml

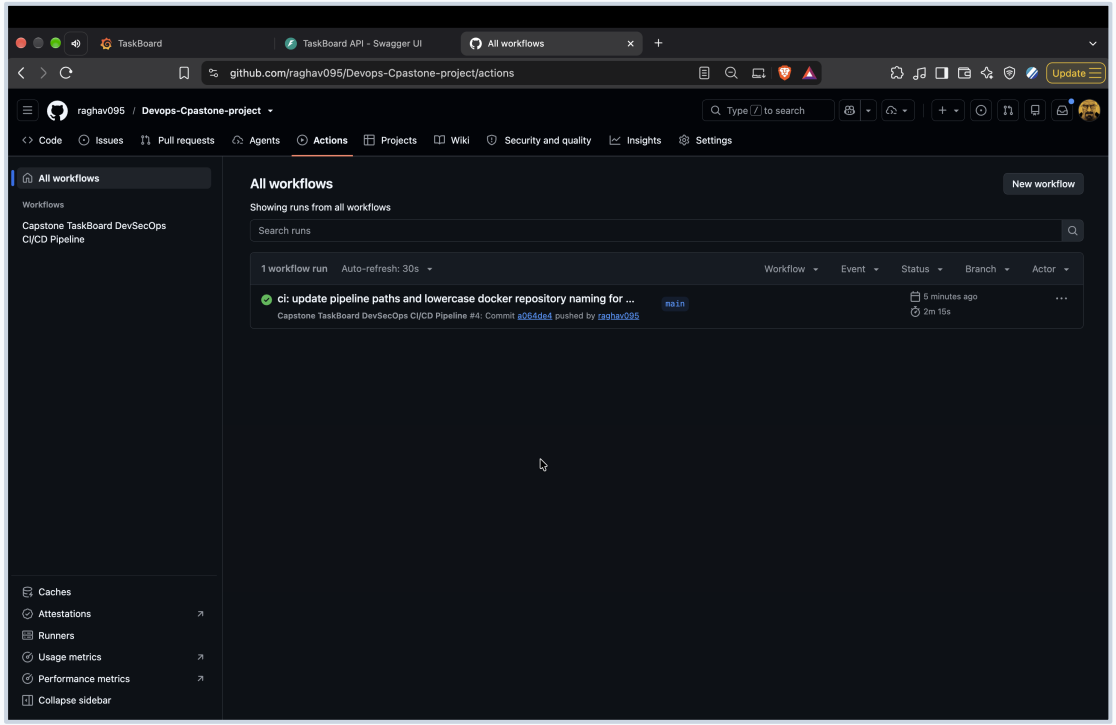
Automated Pipeline Workflow

- Triggered on git push & pull_request to main branch
- Sequential stages: Pytest → Docker Build → Trivy Scan → GHCR Push

Continuous Delivery & Traceability

- Images tagged with dynamic Git commit SHA for immutable releases
- Verified green pipeline runs on GitHub Actions infrastructure



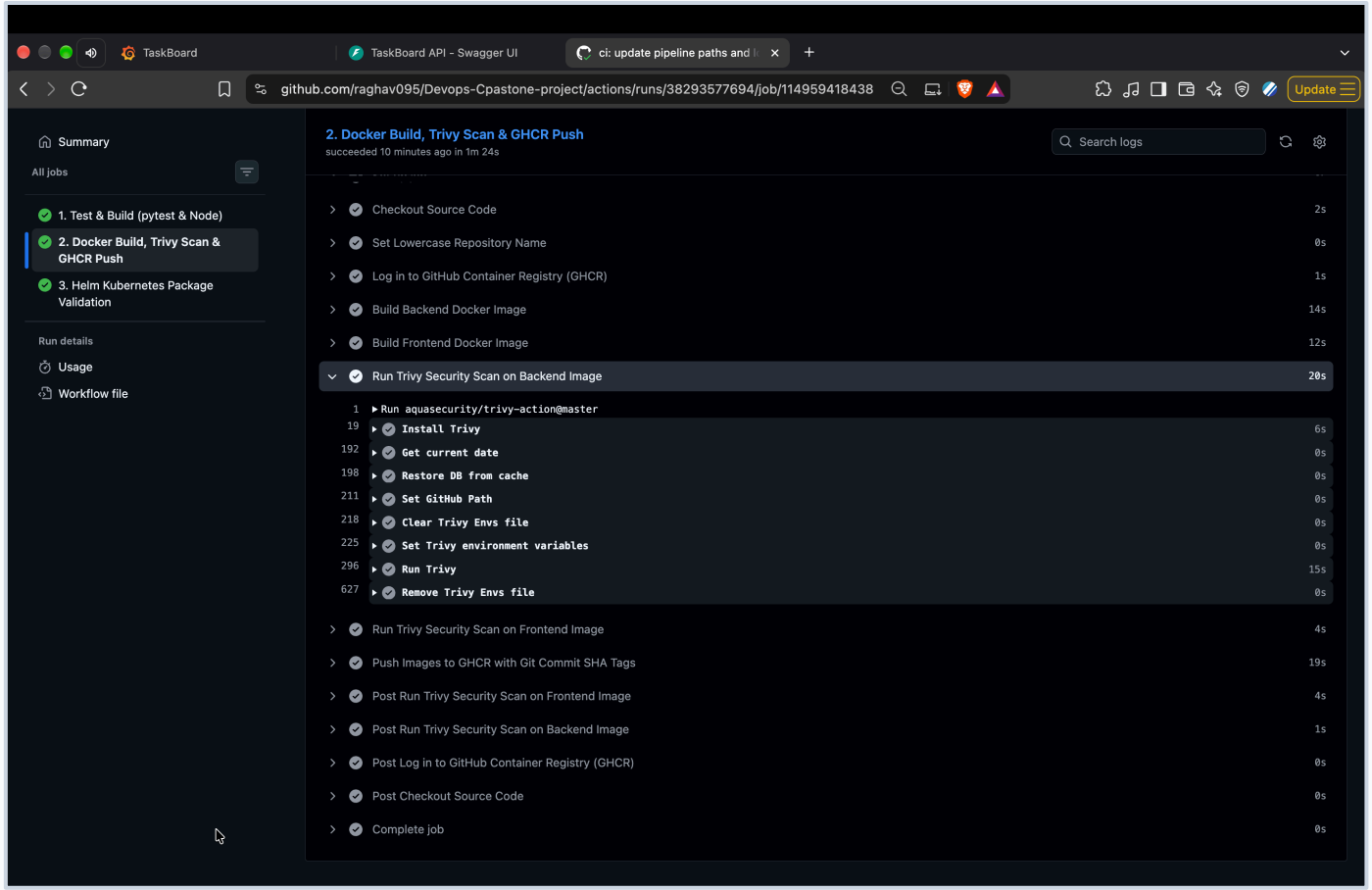


💡 Pipeline file: .github/workflows/devsecops-ci-cd.yml · 100% green build status

6 DevSecOps Vulnerability Scanning (Trivy)

Aqua Security Trivy automated container image security gate in CI pipeline

- **Vulnerability Scan:** Scans base OS packages and application dependencies against NVD
- **Security Gate:** Pipeline automatically fails if CRITICAL or HIGH CVEs detected
- **Non-Root Check:** Enforces non-root container execution and secret hygiene
- **Shift-Left DevSecOps:** Identifies and blocks vulnerabilities prior to registry publication

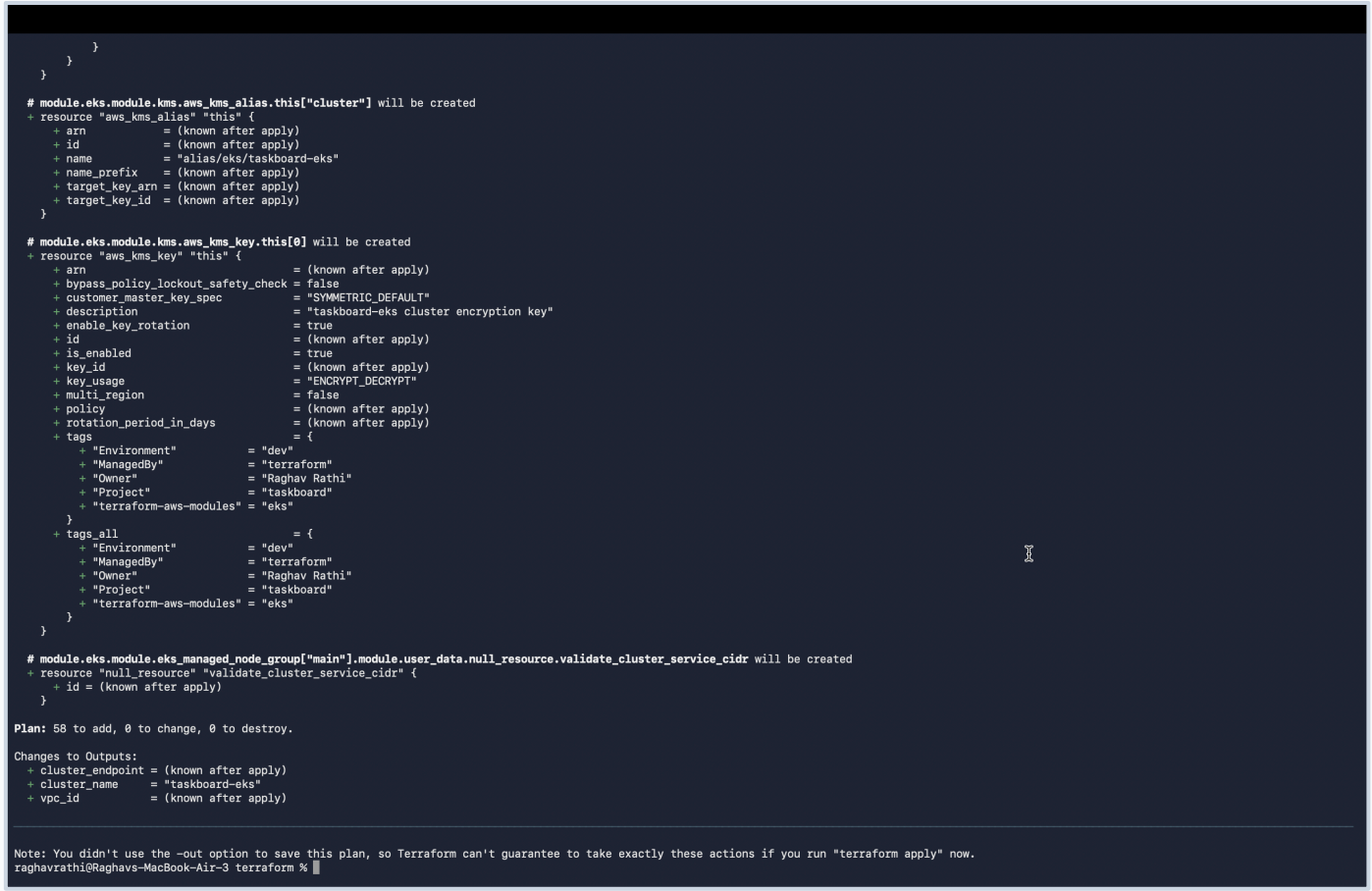


💡 Zero CRITICAL vulnerabilities permitted — images never published without passing Trivy

7 Infrastructure as Code (Terraform — AWS VPC & EKS)

Declarative cloud infrastructure provisioning with full lifecycle management

- **Modular AWS VPC:** 2 public subnets + 2 private subnets across 2 AZs with NAT Gateway
- **AWS EKS Cluster:** Kubernetes 1.31 cluster with managed node group (t3.medium)
- **Terraform Lifecycle:** Complete validate → plan → apply → destroy automated execution
- **Resource Accounting:** 52 cloud resources provisioned and cleanly destroyed



```
        }
      }
  }

  # module.eks.module.kms.aws_kms_alias.this["cluster"] will be created
  + resource "aws_kms_alias" "this" {
      + arn            = (known after apply)
      + id             = (known after apply)
      + name           = "alias/eks/taskboard-eks"
      + name_prefix    = (known after apply)
      + target_key_arn = (known after apply)
      + target_key_id  = (known after apply)
    }

  # module.eks.module.kms.aws_kms_key.this[0] will be created
  + resource "aws_kms_key" "this" {
      + arn                                = (known after apply)
      + bypass_policy_lockout_safety_check = false
      + customer_master_key_spec           = "SYMMETRIC_DEFAULT"
      + description                        = "taskboard-eks cluster encryption key"
      + enable_key_rotation                = true
      + id                                 = (known after apply)
      + is_enabled                         = true
      + key_id                             = (known after apply)
      + key_usage                          = "ENCRYPT_DECRYPT"
      + multi_region                       = false
      + policy                             = (known after apply)
      + rotation_period_in_days            = (known after apply)
      + tags                               = {
          + "Environment"           = "dev"
          + "ManagedBy"             = "terraform"
          + "Owner"                 = "Raghav Rathi"
          + "Project"               = "taskboard"
          + "terraform-aws-modules" = "eks"
        }
      + tags_all                           = {
          + "Environment"           = "dev"
          + "ManagedBy"             = "terraform"
          + "Owner"                 = "Raghav Rathi"
          + "Project"               = "taskboard"
          + "terraform-aws-modules" = "eks"
        }
    }

  # module.eks.module.eks_managed_node_group["main"].module.user_data.null_resource.validate_cluster_service_cidr will be created
  + resource "null_resource" "validate_cluster_service_cidr" {
      + id = (known after apply)
    }

Plan: 58 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + cluster_endpoint = (known after apply)
  + cluster_name     = "taskboard-eks"
  + vpc_id           = (known after apply)


Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
raghavrathi@Raghavs-MacBook-Air-3 terraform %
```

💡 Directory: terraform/ · Declarative modules for VPC, EKS, and security groups

7 AWS Cloud Infrastructure Verification

Real AWS Management Console screenshots proving infrastructure was live and verified

AWS EKS Cluster

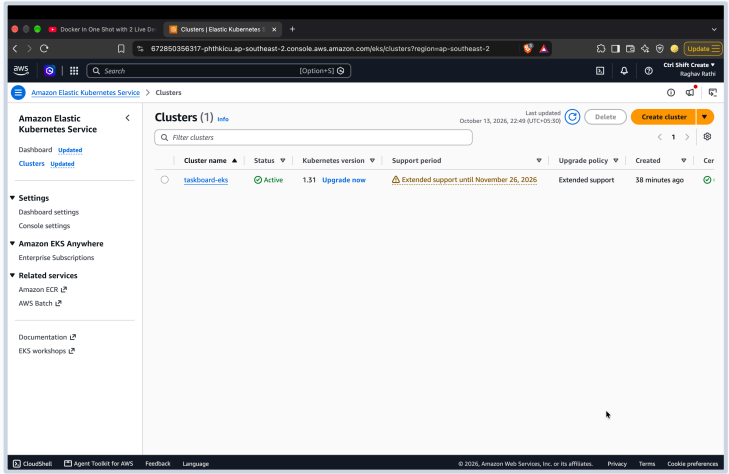
- Active Kubernetes 1.31
- Managed node group healthy

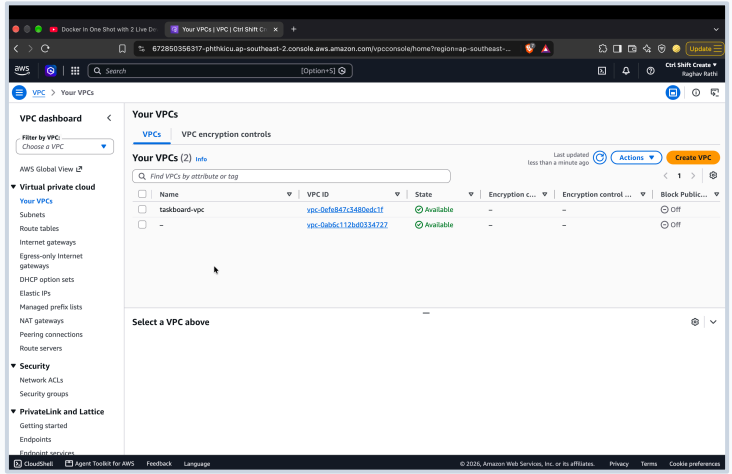
AWS VPC Console

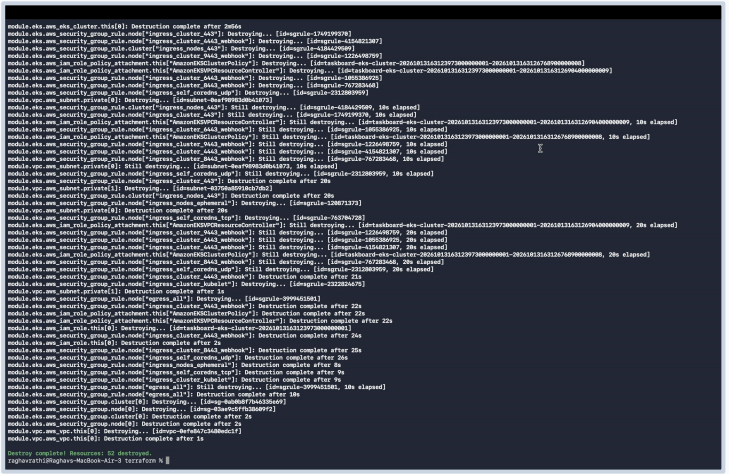
- Public & Private subnets
- NAT Gateway & Route tables

Clean Teardown

- 52 resources destroyed
- Zero lingering AWS costs







Full infrastructure lifecycle executed cleanly with zero cost leakage

8 Kubernetes Deployment & Helm Package Management

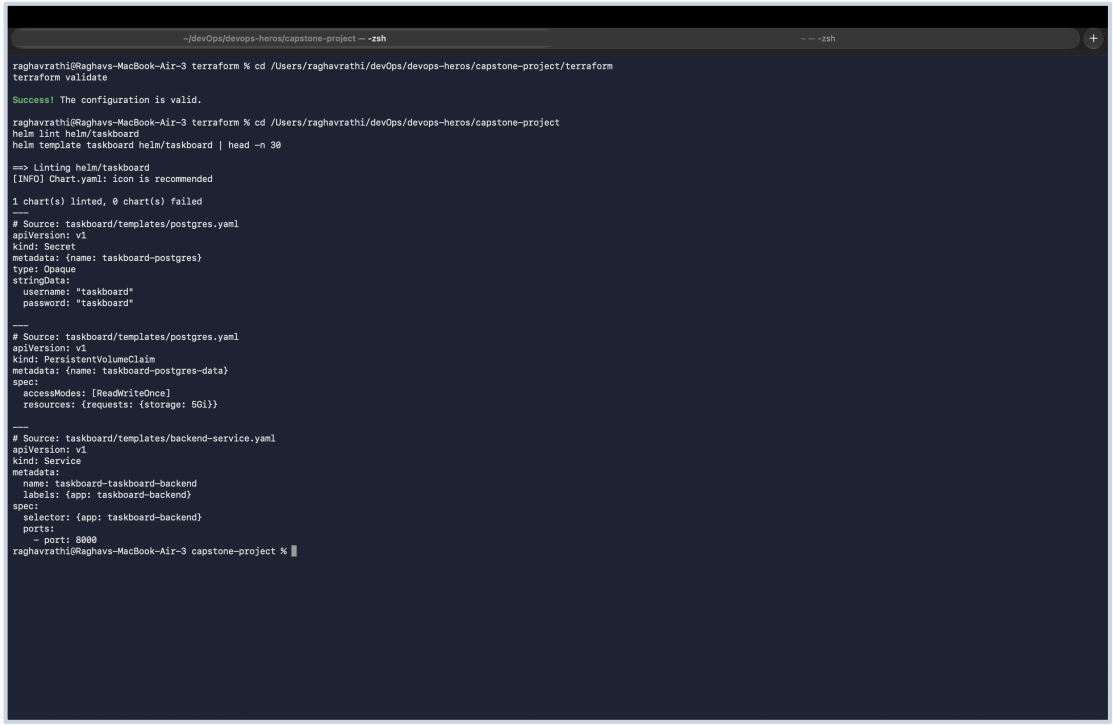
Production orchestration with parameterized Helm 3 chart: helm/taskboard/

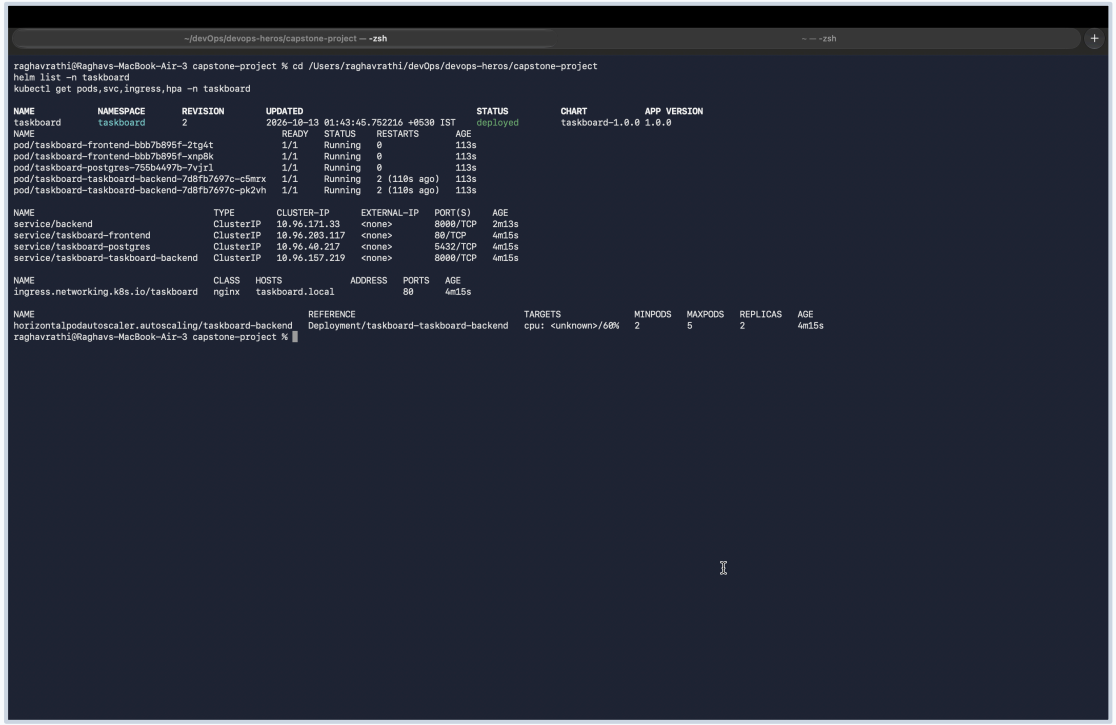
⎈ Helm 3 Chart Packaging

- Modular templates for Deployments, Services, Ingress, and HPA
- Pre-deployment validation via helm lint and helm template

🟢 Running Pods, Services & HPA

- All pods healthy and Running in dedicated taskboard namespace
- Horizontal Pod Autoscaler configured for CPU-based scaling



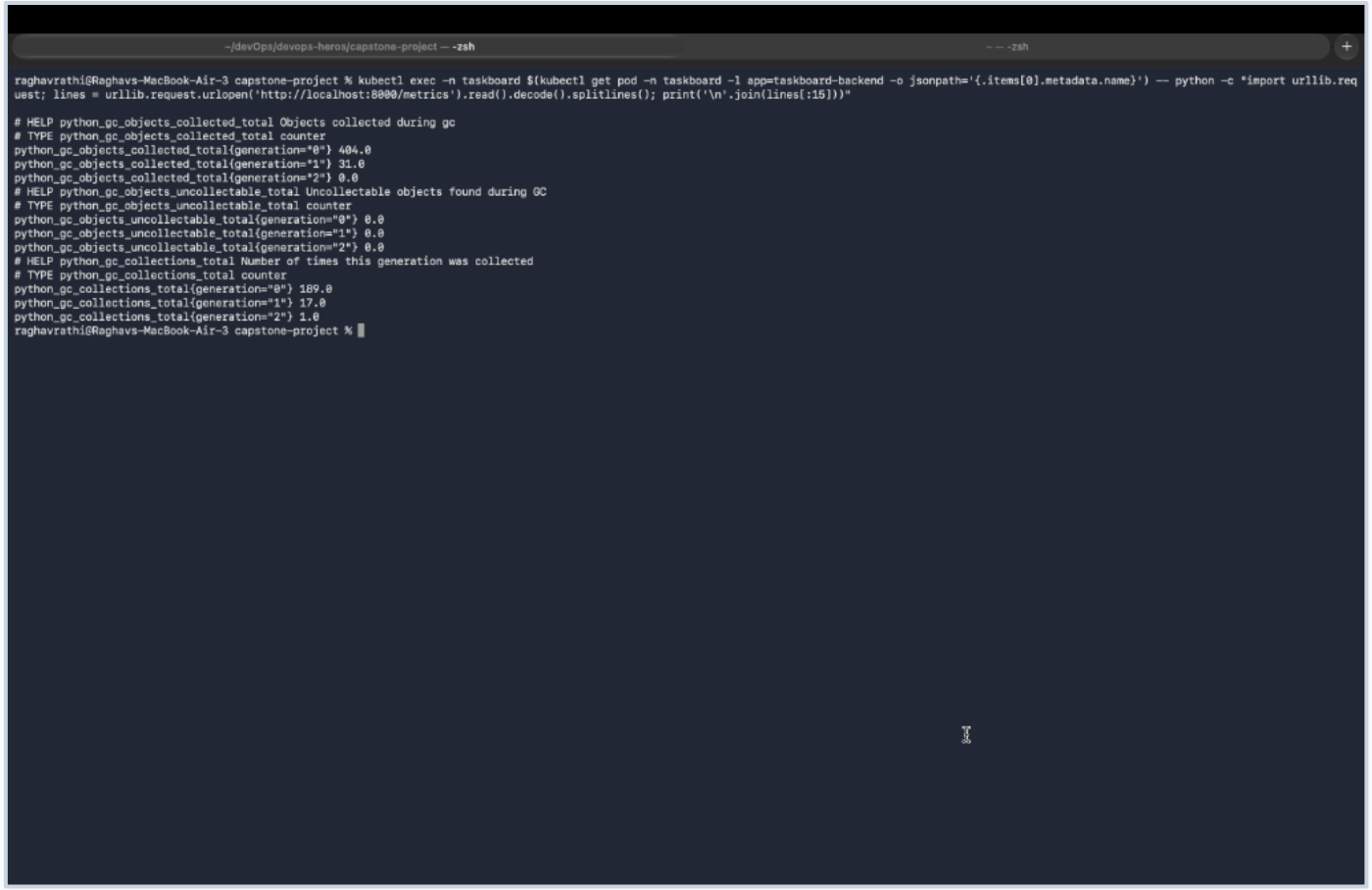


💡 Chart location: helm/taskboard/ · Deployed via Helm release: taskboard

9 Observability — Prometheus Metrics Instrumentation

Real-time application metrics exported at /metrics endpoint

- **FastAPI Instrumentation:** Configured via prometheus-fastapi-instrumentator
- **Exported Metrics:** HTTP request counts, latency histograms, and status codes
- **Prometheus Scraper:** Kubernetes ServiceMonitor auto-discovers and scrapes backend pods
- **Grafana Dashboards:** Pre-configured Prometheus datasource in monitoring/grafana-values.yaml
- **Health Probes:** Liveness (/health) and readiness (/ready) monitored continuously

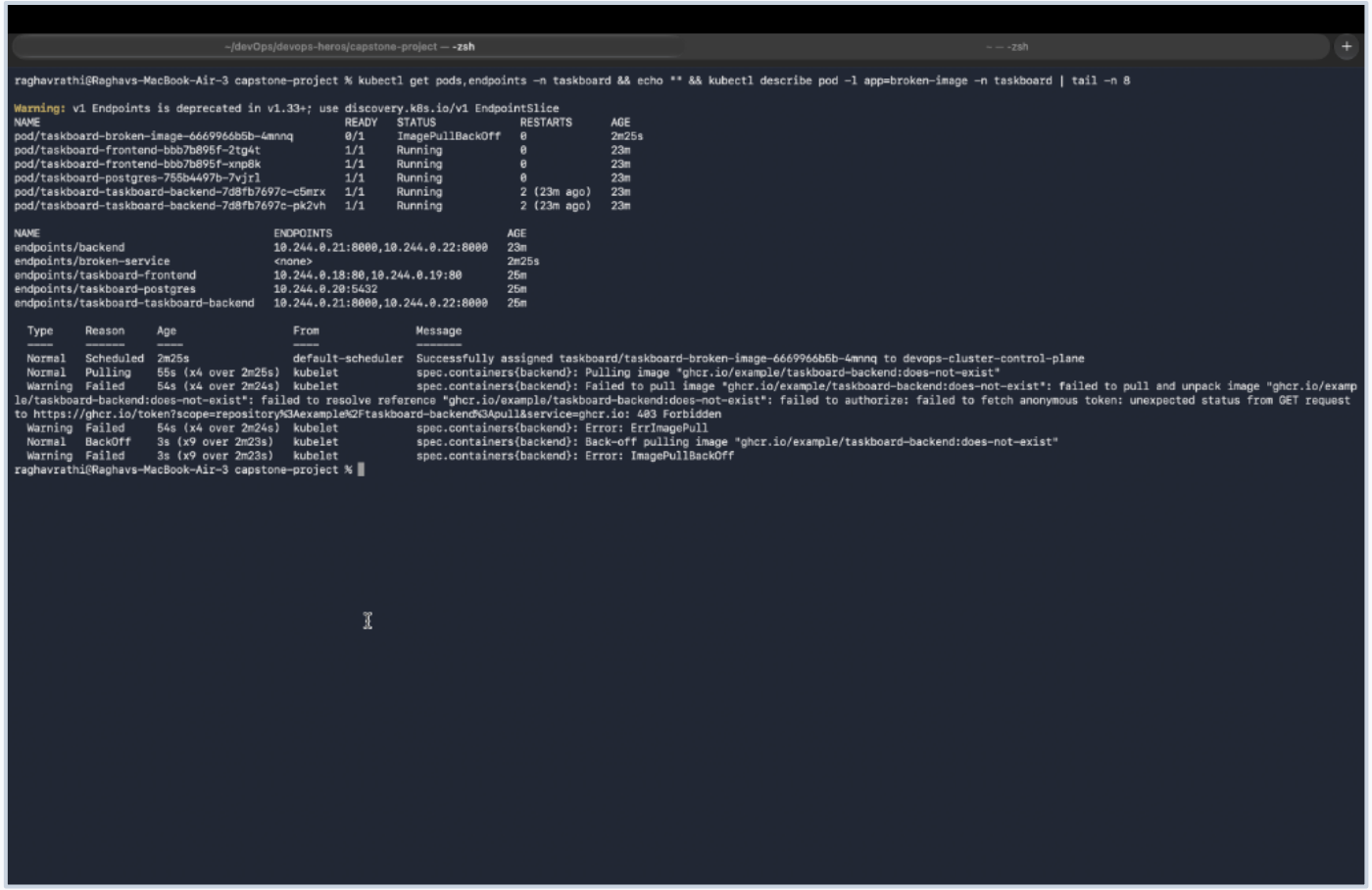


💡 Metrics scrape verified via: curl http://localhost:8000/metrics

10 Incident Response, Troubleshooting & Root Cause Analysis

Hands-on failure diagnosis, remediation, and zero-downtime rollback

- **Scenario 1: ImagePullBackOff:** Diagnosed with kubectl describe pod → Root cause: non-existent tag → Fixed image
- **Scenario 2: Service Error:** Inspected endpoints (<none>) → Root cause: mismatched label selector → Aligned labels
- **Zero-Downtime Rollback:** Instant recovery executed using kubectl rollout undo deployment/<name>
- **Lab Artifacts:** Manifests preserved in troubleshooting/broken-image.yaml & broken-service.yaml



💡 Manifests: troubleshooting/broken-image.yaml & troubleshooting/broken-service.yaml